ASSIGNMENT 1 – TASK 8: Apply Polymorphism

Bloom's Level: K4 – Analyse | Marks: 4

Overloading is compile-time polymorphism: methods share a name but differ in parameter lists. Overriding is run-time polymorphism: the actual object's implementation is chosen when an overridden instance method is called.

```
abstract class Vehicle {
    private final String regNo;

    Vehicle(String regNo) { this.regNo = regNo; }

    public String getRegNo() { return regNo; }

    public abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative minutes");
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Vehicle)) return false;
        Vehicle other = (Vehicle) obj;
        return regNo.equals(other.regNo);
    }

    @Override
    public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override public double ratePerHour() { return 10.0; }
}

class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override public double ratePerHour() { return 30.0; }
}

interface Chargeable {
    double chargingFee(int units);
}

class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double ratePerHour() { return 30.0; }

    @Override
    public double chargingFee(int units) {
        if (units < 0) throw new IllegalArgumentException("Negative units");
        return units * 12.0;
    }
}

public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = {
            new TwoWheeler("TN01A1"),
            new Car("TN02B2"),
```

```
            new ElectricCar("TN03C3")
        };

        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable c) {
                fee += c.chargingFee(10);
            }
            System.out.println(v.getClass().getSimpleName() + " "
                + v.getRegNo() + " -> Rs " + fee);
        }

        System.out.println(new Car("TN02B2").equals(
            new ElectricCar("TN02B2")));
    }
}
```

The interface method name is chargingFee(int units), exactly as specified. The pattern `v instanceof Chargeable c` tests and safely binds the object as Chargeable; it avoids an unnecessary cast to ElectricCar. In the full FeeDemo example, the common Vehicle.calculateFee() method calls ratePerHour(), which is dynamically dispatched.

Dispatch trace	Value
Declared reference type	Vehicle
Actual object	Car
Call made	v.calculateFee(130)
Method executed first	Vehicle.calculateFee(int)
Overridden method selected inside it	Car.ratePerHour()
Billable hours	(130 + 59) / 60 = 3

If a chain of k instanceof checks is used instead, the worst case tests up to k types for each vehicle, so a pass over n vehicles can take O(nk) type-check work. Dynamic dispatch avoids the explicit chain; each method call remains O(1) under this model. The Vehicle[] uses O(n) reference space.

Overloading example: calculateFee(int minutes) and calculateFee(LocalTime entry, LocalTime exit) are selected by their parameter lists at compile time. Ensure the time overload accounts for overnight parking using LocalDateTime or Instant.